

Universidade São Judas Tadeu

Andressa Emily Rabêlo Pereira 823213904
Júlia Oliveira Rocha 823214680
Lucas Marzocca 823116813
Matheus Henrique Ferreira Guimarães 823141914
Marcos Vinicius Santos 82327399
Mylene Soares Rocha 824144075

Projeto Meowlang — Analisador ILéxico

Teoria da Computação e Compiladores

Prof^º Milkes Yone Alvarenga

**São Paulo
2026**

Índice

1. COMPILAÇÃO E INTERPRETAÇÃO.....	3
1.1 Compiladores.....	3
1.2 Interpretadores	3
1.3 Modelos Híbridos e Compilação JIT	4
2. ANÁLISE LÉXICA	4
2.1 Etapas da Análise Léxica	5
2.2 Categorias de Tokens.....	6
3. MECANISMOS ENVOLVIDOS.....	6
3.1 Expressões Regulares.....	7
3.2 Autômatos Finitos	7
3.3 Algoritmos de Conversão	8
3.4 Ferramentas Geradoras de Analisadores Léxicos.....	8
4. TABELA DE SÍMBOLOS.....	9
4.1 Estrutura e Operações.....	9
4.2 Gerência de Escopos	10
6. BIBLIOGRAFIA	12

1. COMPILAÇÃO E INTERPRETAÇÃO

Antes de compreender o que é um analisador léxico, é fundamental distinguir os dois processos pelos quais um programa escrito em linguagem de alto nível pode ser executado por um computador: a compilação e a interpretação. Ambos partem do mesmo código-fonte, mas diferem profundamente na forma como o transformam em ações executáveis (AHO et al., 2007).

1.1 Compiladores

Um compilador é um programa que traduz integralmente o código-fonte de uma linguagem de alto nível para um código de destino, geralmente código de máquina ou código intermediário, antes que qualquer instrução seja executada. O processo ocorre em fases bem definidas e sequenciais: análise léxica, análise sintática, análise semântica, geração de código intermediário, otimização e geração de código final.

A principal vantagem da compilação é o desempenho em tempo de execução: uma vez compilado, o programa roda diretamente sobre o processador sem necessidade de reanálise do código-fonte. O compilador realiza análises estáticas profundas sobre o programa, detectando erros de tipo e de estrutura antes mesmo da execução. Exemplos clássicos de linguagens compiladas incluem C, C++, Rust e Go (COOPER; TORCZON, 2022).

A estrutura clássica de um compilador pode ser dividida em duas grandes partes:

- Front-end: responsável pela análise do código-fonte, engloba a análise léxica, a análise sintática e a análise semântica. O front-end é dependente da linguagem fonte e independente da máquina alvo.
- Back-end: responsável pela síntese, ou seja, pela geração e otimização do código de destino. O back-end é dependente da arquitetura alvo e independente da linguagem fonte.

1.2 Interpretadores

Um interpretador lê o código-fonte instrução a instrução e o executa imediatamente, sem produzir um arquivo de saída traduzido. Cada vez que o programa é executado, o interpretador refaz o processo de análise e execução.

Essa abordagem oferece maior flexibilidade e facilidade de depuração, erros podem ser identificados e corrigidos de forma incremental durante a execução.

Contudo, o desempenho em tempo de execução é inferior ao de um compilador, pois a análise é repetida a cada execução. Python, Ruby e PHP são exemplos de linguagens tradicionalmente interpretadas (SIPSER, 2013).

1.3 Modelos Híbridos e Compilação JIT

Muitas linguagens modernas adotam modelos híbridos para equilibrar as vantagens de ambas as abordagens. Java e C#, por exemplo, compilam o código-fonte para um bytecode intermediário, que é então executado por uma Máquina Virtual (JVM ou CLR). Essa máquina virtual pode interpretar o bytecode diretamente ou utilizar a técnica de Compilação Just-In-Time (JIT).

A compilação JIT é uma técnica que compila trechos de bytecode para código de máquina nativo quando são necessários durante a execução, ao invés de compilar todo o programa antecipadamente. O compilador JIT pode ainda considerar as características específicas do hardware em que está rodando, gerando código mais otimizado do que uma compilação estática poderia produzir (GUALDA; PEREIRA; GUIMARÃES, 2013).

Exemplos de ambientes que utilizam JIT incluem a JVM (Java), o CLR (.NET/C#) e o motor V8 do Google Chrome (JavaScript). O tempo gasto em interpretação pura é geralmente uma ordem de grandeza maior que a execução de código compilado, o que torna o JIT uma estratégia fundamental para linguagens de alto nível que exigem desempenho competitivo.

Independentemente do modelo adotado, o ponto de partida de todo processo de tradução ou interpretação é a análise léxica, a primeira fase do front-end de qualquer compilador ou interpretador.

2. ANÁLISE LÉXICA

A análise léxica, também denominada varredura, tokenização ou scanning, é a primeira fase do processo de compilação ou interpretação. Sua responsabilidade é ler o código-fonte caractere a caractere e agrupar as sequências lidas em unidades significativas denominadas tokens, descartando elementos irrelevantes para as fases posteriores, como espaços em branco e comentários (AHO et al., 2007).

Conforme definido formalmente na literatura, um token é um par:

⟨tipo, valor⟩

onde **tipo** identifica a categoria gramatical do token (palavra-chave, identificador, operador, literal, etc.) e **valor** corresponde ao lexema, a sequência exata de caracteres reconhecida no código-fonte. Em muitos compiladores, a classe do token é representada internamente por um número inteiro para tornar a representação mais compacta (PUC GOIÁS, 2014).

O componente responsável pela análise léxica é denominado analisador léxico (ou lexer / scanner). Ele atua como uma interface entre o código-fonte bruto e as demais fases do compilador, fornecendo ao analisador sintático um fluxo limpo e estruturado de tokens sob demanda. Essa interação é comumente implementada fazendo com que o analisador léxico seja uma sub-rotina chamada pelo parser sempre que um novo token é necessário.

A separação entre análise léxica e sintática é justificada por motivos práticos importantes: os tokens podem ser descritos por notações simples como expressões regulares, enquanto a estrutura sintática requer gramáticas livres de contexto, mais expressivas e computacionalmente mais custosas. Reconhecedores construídos a partir de expressões regulares são mais eficientes e compactos (WIKIVERSIDADE, 2024).

2.1 Etapas da Análise Léxica

O processo de análise léxica segue uma sequência de etapas bem definidas:

- Leitura do código-fonte: o analisador percorre o texto de entrada caractere a caractere, mantendo o controle da posição atual (linha e coluna) para fins de reporte de erros léxicos.
- Eliminação de espaços e quebras de linha: espaços em branco, tabulações e quebras de linha são tratados como separadores e ignorados pelo lexer na maioria das linguagens de programação.
- Remoção de comentários: comentários de linha e de bloco são identificados pelos seus delimitadores e descartados antes do processamento dos demais tokens.
- Reconhecimento de padrões: o analisador aplica regras — expressões regulares ou autômatos finitos — para identificar e classificar cada sequência de caracteres como um token válido.

- Reporte de erros léxicos: sequências de caracteres que não correspondem a nenhum padrão válido são sinalizadas como erros léxicos, indicando linha e coluna da ocorrência.

2.2 Categorias de Tokens

Os tokens reconhecidos por um analisador léxico são organizados em categorias. A tabela abaixo apresenta as principais:

Tipo	Descrição	Exemplos
KEYWORD	Palavras reservadas da linguagem	if, while, return, class, for
IDENTIFICADOR	Nomes de variáveis, funções e classes definidos pelo programador	contador, calcular, resultado
LITERAL INTEIRO	Valores numéricos inteiros	0, 42, -7, 255
LITERAL FLOAT	Valores numéricos de ponto flutuante	3.14, 2.718, 0.001
LITERAL STRING	Sequências de texto entre aspas	"Olá Mundo", 'texto'
LITERAL BOOLEANO	Valores lógicos	true, false
OPERADOR	Operadores aritméticos, relacionais e lógicos	+ - * / == != <= >= &&
DELIMITADOR	Pontuação estrutural	() {} [] , ; .
COMENTÁRIO	Texto ignorado pelo compilador	// linha, /* bloco */
ERRO	Token inválido não reconhecido	@, \$\$, ##invalido

3. MECANISMOS ENVOLVIDOS

A implementação de um analisador léxico fundamenta-se em dois modelos formais da teoria da computação: expressões regulares e autômatos finitos. A relação entre esses dois formalismos é central para entender como um lexer funciona internamente, e foi estabelecida formalmente por Kleene (1956), que demonstrou a equivalência entre linguagens regulares, expressões regulares e autômatos finitos (HOPCROFT; MOTWANI; ULLMAN, 2006).

3.1 Expressões Regulares

Uma expressão regular (ER) é uma notação compacta para descrever padrões de strings sobre um alfabeto. Na análise léxica, cada categoria de token é especificada por uma expressão regular. As operações fundamentais são:

- **Concatenação (AB):** A seguido de B.
- **União (A|B):** A ou B.
- **Fechamento de Kleene (A*):** zero ou mais ocorrências de A.

A partir dessas operações básicas derivam-se atalhos como A+ (uma ou mais ocorrências), A? (zero ou uma ocorrência) e classes de caracteres como [a-z] (qualquer letra minúscula). Exemplos práticos de expressões regulares para tokens comuns incluem:

- **Identificadores:** [a-zA-Z_][a-zA-Z0-9_]* — Começa com letra ou underscore, seguido de letras, dígitos ou underscores.
- **Inteiros:** [0-9]+ — Um ou mais dígitos.
- **Números decimais:** [0-9]+(\.[0-9]+)? — Parte inteira obrigatória, parte decimal opcional.
- **Strings:** "[^"]*" — Qualquer sequência de caracteres entre aspas duplas.

A implementação do analisador léxico por meio de uma tabela de transições, que indica a passagem de um estado a outro pela leitura de um determinado caractere, é diretamente derivada das expressões regulares e dos autômatos que as reconhecem (WIKIVERSIDADE, 2024).

3.2 Autômatos Finitos

Toda expressão regular pode ser convertida em um autômato finito equivalente, e essa conversão é o que os geradores de lexers executam internamente. Existem dois tipos fundamentais (HOPCROFT; MOTWANI; ULLMAN, 2006):

- **Autômato Finito Não Determinístico (AFND):** construído diretamente a partir da expressão regular. Permite múltiplas transições para o mesmo símbolo de entrada a partir de um mesmo estado, além de transições espontâneas (ϵ -transições). É mais simples de construir, porém ineficiente para execução direta por requerer a simulação de múltiplos estados simultâneos.
- **Autômato Finito Determinístico (AFD):** cada estado possui exatamente uma transição definida para cada símbolo do alfabeto — sem ambiguidade. É eficiente para execução, pois o lexer percorre o AFD em tempo linear sobre o comprimento da entrada, sem retrocesso (backtracking).

3.3 Algoritmos de Conversão

O processo completo de construção de um lexer a partir de expressões regulares envolve os seguintes algoritmos formais (AHO et al., 2007; COOPER; TORCZON, 2022):

- Algoritmo de Thompson (Construção de Thompson / McNaughton-Yamada-Thompson): converte uma expressão regular em um AFND de forma sintático-dirigida, construindo recursivamente o autômato a partir das subexpressões. Cada operador (concatenação, união, Kleene) possui um template de construção correspondente.
- Construção de Subconjuntos (Powerset Construction): converte o AFND em um AFD equivalente. A ideia central é que cada estado do AFD construído corresponde a um subconjunto dos estados do AFND. O resultado é um AFD que aceita exatamente a mesma linguagem que o AFND original.
- Algoritmo de Minimização de Hopcroft: minimiza o AFD, reduzindo o número de estados ao mínimo necessário sem alterar a linguagem aceita. O resultado é um AFD mínimo, com tabelas de transição mais compactas e eficientes.

O pipeline completo pode ser representado como: Expressão Regular → AFND (Thompson) → AFD (Subconjuntos) → AFD Mínimo (Hopcroft) → Tabela de Transições → Código do Lexer. Este é o processo executado internamente pelas ferramentas geradoras de analisadores léxicos (SCIENCEDIRECT, 2024).

3.4 Ferramentas Geradoras de Analisadores Léxicos

Na prática, raramente se constrói o autômato manualmente para cada nova linguagem. Utilizam-se ferramentas que automatizam a geração do analisador léxico a partir de especificações em expressões regulares. Conforme descreve a literatura (UNESP, 2014; COMPILADORES PARA HUMANOS, 2020):

Ferramenta	Linguagem de Saída	Descrição
Lex / Flex	C / C++	Ferramentas clássicas do ambiente Unix; Flex é a versão moderna e livre do Lex original
ANTLR	Java, Python, C#, Go, outros	Gera lexer e parser integrados a partir de gramáticas combinadas
JFlex	Java	Gerador de lexers para a plataforma Java, com suporte a Unicode
PLY (Python Lex-Yacc)	Python	Implementação pura em Python inspirada no Lex/Yacc

re2c	C / C++	Gera código C altamente otimizado a partir de expressões regulares
JavaCC	Java	Gerador de lexers e parsers para Java, utilizado em projetos acadêmicos e industriais

4. TABELA DE SÍMBOLOS

A tabela de símbolos é uma estrutura de dados central em um compilador ou interpretador. Ela pode ser definida como um tipo de dado abstrato que mapeia identificadores, denominados símbolos, à sua informação semântica associada, atuando como um dicionário para o processo de tradução (ALCÂNTARA, 2025).

Embora seja alimentada inicialmente pela análise léxica, que cria as primeiras entradas ao encontrar identificadores no código-fonte, a tabela de símbolos persiste e é enriquecida ao longo das fases seguintes. O analisador léxico contribui registrando o lexema e a posição de cada identificador; o analisador sintático adiciona informações de escopo e estrutura; o analisador semântico acrescenta tipos e verifica consistência (AHO et al., 2007).

É importante distinguir entre identificador e símbolo: um identificador é uma cadeia de caracteres (o lexema), como x. Um símbolo, por outro lado, é um nome único dentro de um contexto específico de escopo. Um mesmo identificador pode representar múltiplos símbolos distintos dependendo do escopo em que aparece (ALCÂNTARA, 2025).

4.1 Estrutura e Operações

A tabela de símbolos é tipicamente implementada como uma tabela hash, garantindo operações de inserção, busca e remoção em tempo $O(1)$ amortizado. Outras estruturas possíveis incluem listas lineares (simples, mas de desempenho pobre para consultas frequentes) e árvores binárias de busca (que permitem travessias ordenadas). Para grandes arquivos de entrada, o armazenamento eficiente é fundamental (MATTA, 2019; AHO et al., 2007).

Cada entrada da tabela contém, entre outros campos:

- Nome (lexema): a sequência de caracteres que identifica o símbolo no código-fonte.
- Tipo: o tipo de dado associado ao símbolo — inteiro, float, string, booleano, função, classe, etc.
- Escopo: o nível de visibilidade do símbolo, global, local a uma função ou a um bloco.
- Valor: o valor atual da variável, quando disponível em tempo de compilação.
- Posição: linha e coluna no código-fonte onde o símbolo foi declarado, para fins de diagnóstico de erros.
- Atributos extras: para funções, o número e os tipos dos parâmetros; para arrays, o tamanho e o tipo dos elementos; para classes, os membros e métodos.

A tabela a seguir ilustra um exemplo simplificado de entradas em uma tabela de símbolos:

Nome	Tipo	Escopo	Valor	Linha
contador	int	global	0	3
nome	string	global	"Garfield"	4
ativo	bool	global	true	5
calcular	function	global	—	8
resultado	float	local (calcular)	3.14	10

4.2 Gerência de Escopos

Em linguagens com escopo léxico (block-structured languages), a tabela de símbolos é organizada como uma pilha de escopos ou como uma lista encadeada de tabelas: ao entrar em um novo bloco ou função, uma nova tabela é empilhada; ao sair, ela é desempilhada. A busca percorre a pilha do topo para a base, respeitando a regra de que declarações internas sombreiam declarações externas de mesmo nome (SLIDESHARE, 2024).

A gerência correta da tabela de símbolos é fundamental para a detecção de erros semânticos como: uso de variável não declarada, redeclaração de identificadores no mesmo escopo, incompatibilidade de tipos em expressões e chamadas de função com número incorreto de argumentos. Segundo a Wikipedia

(2021), a tabela de símbolos é inserida pelos analisadores léxico e sintático e consultada pelo analisador semântico e pelo gerador de código.

5. CONCLUSÃO

A análise léxica, embora seja apenas a primeira fase de um compilador ou interpretador, desempenha um papel estrutural indispensável em todo o processo de tradução de linguagens de programação. Ao transformar um fluxo contínuo de caracteres em uma sequência organizada de tokens, o lexer estabelece a fundação sobre a qual todas as fases subsequentes, sintática, semântica e de geração de código, se constroem.

Os mecanismos formais que sustentam essa fase, expressões regulares e autômatos finitos, demonstram como conceitos abstratos da teoria da computação encontram aplicação direta e eficiente em ferramentas do mundo real. A equivalência entre esses formalismos, estabelecida pelos algoritmos de Thompson, de subconjuntos e de minimização de Hopcroft, é o alicerce teórico que permite a geração automática de lexers eficientes a partir de especificações de alto nível.

A distinção entre compilação e interpretação contextualiza o papel do analisador léxico em diferentes arquiteturas de processamento de linguagens. A compilação JIT, por sua vez, demonstra como os modelos se combinam para atingir desempenho próximo ao de código nativo sem abrir mão da portabilidade dos modelos baseados em bytecode.

Por fim, a tabela de símbolos ilustra como as informações coletadas pela análise léxica transcendem essa fase inicial, sendo utilizadas e enriquecidas ao longo de todo o processo de compilação para garantir a correção semântica do programa. Compreender esses fundamentos não apenas habilita o desenvolvimento de novas linguagens e ferramentas de análise, mas aprofunda o entendimento sobre o funcionamento de todos os compiladores e interpretadores modernos.

6. BIBLIOGRAFIA

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compiladores: Princípios, Técnicas e Ferramentas*. 2. ed. São Paulo: Pearson Addison-Wesley, 2007. ISBN 978-0-321-48681-3. [Título original: *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson Education, 2006].

COOPER, Keith D.; TORCZON, Linda. *Engineering a Compiler*. 3. ed. Cambridge: Morgan Kaufmann/Elsevier, 2022. ISBN 978-0-12-815412-0. Disponível em: <https://www.sciencedirect.com/book/9780128154120>. Acesso em: 21 maio 2026.

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introduction to Automata Theory, Languages, and Computation*. 3. ed. Boston: Addison-Wesley/Pearson, 2006. ISBN 0-321-45536-3.

SIPSER, Michael. *Introduction to the Theory of Computation*. 3. ed. Boston: Cengage Learning, 2013. ISBN 978-1-133-18779-0.

GUALDA, Luiz A. J.; PEREIRA, Márcio B.; GUIMARÃES, Edgar. *Compilação Just-In-Time: Histórico, Arquitetura, Princípios e Sistemas*. ResearchGate, 2013. Disponível em: <https://www.researchgate.net/publication/332585327>. Acesso em: 21 maio 2026.

MATTA, Jad. *Hash table implementation strategies in Symbol Table using JAVA and C++*. Academia.edu / ResearchGate, 2019. Disponível em: <https://www.researchgate.net/publication/338104054>. Acesso em: 21 maio 2026.

UNIVERSIDADE ESTADUAL PAULISTA (UNESP) — IBILCE. *Introdução à Teoria dos Compiladores — Capítulo 2: Analisador Léxico*. Disponível em: <https://www.dcce.ibilce.unesp.br/~aleardo/cursos/compila/cap02.pdf>. Acesso em: 21 maio 2026.

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS (PUC GOIÁS). *Compiladores — Material Didático de Análise Léxica*. Disponível em: https://professor.pucgoias.edu.br/SiteDocente/admin/arquivosUpload/17389/material/Texto_Parte1.pdf. Acesso em: 21 maio 2026.

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL (UFRGS). *Compiladores — Análise Lexical: Expressões Regulares*. Disponível em:

<https://www.inf.ufrgs.br/~nicolas/pdf/Compiladores02-er.pdf>. Acesso em: 21 maio 2026.

UNIVERSIDADE DO PORTO (DCC/FCUP) — VASCONCELOS, Pedro. *Compiladores (CC3001) — Aula 2: Análise Lexical*. 2020. Disponível em: <https://www.dcc.fc.up.pt/~pbv/aulas/compiladores/teoricas/aula02.pdf>. Acesso em: 21 maio 2026.

WIKIVERSIDADE. *Introdução à Teoria dos Compiladores/Análise Léxica*. Disponível em:

https://pt.wikiversity.org/wiki/Introdução_à_Teoria_dos_Compiladores/Análise_Léxica. Acesso em: 21 maio 2026.

WIKIPEDIA. *Tabela de símbolos*. Disponível em: https://pt.wikipedia.org/wiki/Tabela_de_símbolos. Acesso em: 21 maio 2026.

WIKIPEDIA. *Thompson's construction*. Disponível em: https://en.wikipedia.org/wiki/Thompson%27s_construction. Acesso em: 21 maio 2026.

ALCÂNTARA, Frank. *Tabela de Símbolos em Compiladores Modernos — Linguagens Formais e Autômatos*. 2025. Disponível em: <https://frankalcantara.com/lf/11-tabelaSimbolos.html>. Acesso em: 21 maio 2026.

COMPILADORES PARA HUMANOS. *Análise Léxica*. Disponível em: <https://johnidm.gitbooks.io/compiladores-para-humanos/content/part1/lexical-analysis.html>. Acesso em: 21 maio 2026.

SCIENCEDIRECT. *Subset Construction*. Disponível em: <https://www.sciencedirect.com/topics/computer-science/subset-construction>. Acesso em: 21 maio 2026.